Table of Contents

1. Introduction

Capture The Flag (CTF) environments are widely used for cyber security training because they provide controlled, hands-on scenarios that simulate real vulnerabilities. This report documents the design and implementation of a custom CTF virtual machine built around "New Trends in Cyber Security". Instead of only demonstrating single vulnerabilities, the VM includes realistic misconfigurations and insecure design patterns that reflect common security failures in modern systems such as weak identity boundaries, exposed internal services, insecure cryptographic key handling, and privilege escalation from poor operational hardening.

The VM contains **five categories**, each with **two challenges** (medium and high), resulting in a total of **ten flags**. Each challenge is designed to be solvable using common attacker workflows (reconnaissance → enumeration → exploitation → validation), and each flag provides proof that the vulnerability was successfully exploited.

CTF BOX Link -

https://apiitlk-my.sharepoint.com/:f:/g/personal/cb013270_students_apiit_lk/IgAbKbNSP8P1QqQ0AuGBj6NJASF8Vs0PZj8hRxvxlqNnO14?e=8HFcWD

2. System Overview

2.1 Base Operating System

The CTF box is implemented on **Ubuntu 20.04.6 LTS Desktop**. Ubuntu was selected because it is widely used for server workloads, stable over long periods, and supports a large ecosystem of packages. This makes the lab closer to real-world Linux web servers and internal service deployments.

2.2 Installed Services and Tools

The VM includes the following core components:

- **Apache2 (HTTP)**
  Hosts the main landing page and the CTF portal pages. Apache was selected

because it is commonly deployed in production environments and is a realistic target for web vulnerability testing.

- **PHP**
  Used to build intentionally vulnerable web applications and support dynamic challenge pages.
- **OpenSSH Server**
  Used for remote login for privilege escalation and post-exploitation stages.
- **Custom TCP Services (Python-based)**
  Two services were created to simulate misconfigured internal applications exposed over the network.
- **System scheduling (cron + system scripts)**
  Used to simulate insecure operational automation leading to privilege escalation.
- **Forensics utilities (RAR/GIF workflow + common analysis tools)**
  Challenge artifacts were created using image formats, archives, and embedded payloads to simulate incident-response style evidence.

2.3 Network Setup

The VM is configured on a **Host-Only network** to allow safe testing from a Kali Linux attacker machine without exposing vulnerable services to an external LAN. The CTF VM address used during testing was:

- **CTF VM IP:** 192.168.56.105

Host-only networking ensures:

- Isolation from the public internet
- Repeatable access from the attacker VM
- No risk of scanning or exploitation affecting external systems

2.4 Access Methods

The box is accessed through:

- **Web browser (HTTP)**: http://192.168.56.105/ and http://192.168.56.105/ctf/
- **SSH**: ssh ctfuser@192.168.56.105
- **Raw TCP connections** (netcat): ports **4444** and **9001**

3. Challenge Description

This section describes each challenge, including the category, difficulty, vulnerability explanation, and flag capture method.

3.1 Web Application Exploitation

1) Web Medium – SQL Injection (Authentication Bypass)

Difficulty: Medium
Concept: Input is inserted into an SQL query without parameterized statements.

Technical explanation:
The login endpoint builds an SQL query using unsensitized user input. Because the user-controlled fields are not parameterized, an attacker can inject SQL logic to alter the query outcome. A common result is authentication bypass, where the query returns a valid row even without correct credentials. This mirrors real issues seen in legacy PHP applications where string concatenation is used instead of prepared statements.

Flag capture method:
After bypassing authentication, the web application reveals a protected page containing internal data and the flag (e.g., a "notes" or "dashboard" section visible only to authenticated users).

2) Web High – Stored XSS (Admin Context Execution)

Difficulty: High
Concept: Untrusted input is stored and later rendered without output encoding.

Technical explanation:
The application contains a form (e.g., feedback/reporting) where user input is stored server-side and displayed to an administrator panel. Because output is not properly HTML-encoded (e.g., missing htmlspecialchars()), attacker-controlled JavaScript executes when the admin loads the page. Stored XSS is high impact because it can compromise sessions, pivot to other endpoints, and enable actions performed as the victim (administrator).

Flag capture method:
When the malicious payload runs in the admin view, it enables access to an admin-only location where the high-level web flag is stored (e.g., admin review content or a hidden endpoint only visible to admins).

3.2 Cryptography

1) Crypto Medium – Vigenère Cipher Decryption (Key Hint Provided)

Difficulty: Medium
Concept: Classical polyalphabetic cipher with a repeated key.

Technical explanation:
A message file is encrypted using a Vigenère cipher. The encryption shifts each character based on the corresponding key character. The system provides a subtle hint for the key within the web content (for example, in HTML comments). This is designed to be medium difficulty because the solver must recognize the cipher type and apply correct decryption, typically using a script or known method.

Flag capture method:
After applying Vigenère decryption, the plaintext reveals the crypto medium flag.

2) Crypto High – AES-Encrypted Backup (Weak Key Management)

Difficulty: High
Concept: Strong encryption can still fail if key derivation is predictable.

Technical explanation:
An archive contains an AES-256-CBC encrypted file (e.g., flag.enc). The encryption algorithm is modern and strong; however, the key is derived from predictable operational metadata (project code, deployment year, and service user name). This simulates a real-world key management failure where secrets are assembled from public or easily discoverable information, making decryption feasible without brute force.

To avoid circular logic, the archive itself is not password-protected; instead, the solver must decrypt the encrypted file using a reconstructed key.

Flag capture method:
The solver derives the key using system artefacts and then runs AES decryption using OpenSSL. The decrypted file contains the crypto high flag.

3.3 Network Exploitation

1) Network Medium – Service Banner Disclosure (Port 4444)

Difficulty: Medium
Concept: Misconfigured internal service leaks sensitive info.

Technical explanation:
A TCP service on port 4444 prints sensitive information immediately upon connection. This reflects a real misconfiguration where a service exposes debug banners, secrets, or internal identifiers. Since the service sends output without authentication, service discovery tools (e.g., nmap -sV) may capture and display the banner automatically.

Flag capture method:
Connecting to port 4444 returns the flag directly as part of the banner output.

2) Network High – Unauthenticated Debug Command (Port 9001)

Difficulty: High
Concept: Internal diagnostic features exposed to attackers.

Technical explanation:
The TCP service on port 9001 behaves like an internal diagnostic interface. It provides commands such as STATUS and DEBUG without access controls. Although the banner does not reveal the flag immediately, invoking the debug functionality exposes internal diagnostic data that includes the flag. This simulates internal services mistakenly exposed to untrusted networks, a common issue in cloud environments and flat networks.

Flag capture method:
The solver sends a valid debug command to the service and captures the flag from the response.

3.4 Forensics

1) Forensics Medium – PCAP Analysis (Reconstruct Sensitive Content)

Difficulty: Medium
Concept: Network forensic reconstruction.

Technical explanation:
A packet capture file contains traffic where sensitive information is exposed. The challenge requires using Wireshark (or similar tools) to follow a TCP stream and recover the hidden flag from payload data. This simulates real-world incident response where analysts investigate unencrypted or improperly logged network activity.

Flag capture method:
The flag is recovered from the reconstructed TCP stream view.

2) Forensics High – Polyglot Evidence File (PNG + Embedded RAR + GIF Frames)

Difficulty: High
Concept: Multi-layer forensic recovery and decoding.

Technical explanation:
The challenge provides a file that appears to be a normal PNG image (evidence.png). However, binary inspection reveals a hidden RAR archive embedded within the file. The solver must:

1. Identify an embedded archive signature (e.g., Rar!)
2. Carve the archive from the image at the correct offset
3. Extract the archive contents to obtain a GIF
4. Decompose the GIF into frames
5. Locate a QR code frame that contains encoded data
6. Decode the QR content and then decode Base64 to recover the flag

This represents realistic forensic work: file signature analysis, data carving, nested container extraction, and decoding hidden payloads.

Flag capture method:
The final decoded payload reveals the forensics high flag.

3.5 Privilege Escalation

1) PrivEsc Medium – Sudo Misconfiguration (Abusing Allowed Root Command)

Difficulty: Medium
Concept: Over-permissive sudo rules.

Technical explanation:
A low-privileged user is allowed to run a specific program as root without entering a password. Although the program appears harmless, it can be used to access privileged files. This mirrors real operational mistakes where sudoers entries are written too broadly or without considering tool abuse.

Flag capture method:
By executing the allowed command as root, the solver reads a root-only flag file.

2) PrivEsc High – PATH Hijacking via Root Cron Job

Difficulty: High
Concept: Insecure automation + environment manipulation.

Technical explanation:
A scheduled cron job runs as root and executes a script that calls common system utilities without absolute paths (e.g., calling tar rather than /bin/tar). Because the job environment includes a user-controlled directory early in the PATH, an attacker can place a malicious executable with the same name. When cron runs, root executes the attacker's program, resulting in privilege escalation.

This reflects a real-world failure in operational hardening: insecure PATH, unsafe scripting practices, and privileged scheduled tasks.

Flag capture method:
After cron executes, a root-owned output file is created in a world-readable location containing the flag.

4. Risk & Threat Analysis

This section maps each vulnerability to realistic impact and risk.

4.1 Web Exploitation

- **SQL Injection (Medium):**
  Impact: Authentication bypass, data exposure, potential database modification.
  Risk level: High (remote, low complexity, common in legacy apps).
- **Stored XSS (High):**
  Impact: Session compromise, admin impersonation, data modification, persistent exploitation.
  Risk level: High to Critical depending on admin privileges and session controls.

4.2 Cryptography

- **Vigenère Cipher (Medium):**
  Impact: Disclosure of "protected" information due to weak classical encryption.
  Risk level: Medium, but demonstrates insecure custom cryptography patterns.
- **AES with predictable key (High):**
  Impact: Strong encryption undermined by weak key management.
  Risk level: High; key management failures are common in real deployments.

4.3 Network Exploitation

- **Banner leak (Medium):**
  Impact: Information disclosure, secrets exposed during reconnaissance.
  Risk level: Medium, often an enabler for later attacks.
- **Debug service (High):**
  Impact: Internal data disclosure, potential stepping stone to further compromise.
  Risk level: High, especially in flat networks/cloud environments.

4.4 Forensics

- **PCAP reconstruction (Medium):**
  Impact: Sensitive data can be recovered from network captures or unencrypted flows.
  Risk level: Medium to High depending on the captured content.
- **Polyglot evidence file (High):**
  Impact: Demonstrates how attackers can hide payloads and exfiltrate data inside "normal" files.
  Risk level: High; malware campaigns frequently use layered packing/encoding.

4.5 Privilege Escalation

- **Sudo misconfiguration (Medium):**
  Impact: Access to root-only resources, possible system takeover.
  Risk level: High; this is a frequent real-world admin error.
- **PATH hijack cron (High):**
  Impact: Full root compromise, persistence, data destruction, lateral movement.
  Risk level: Critical because it results in full system control.

5. Incident Response & Recovery

5.1 Detection

- Enable and review Apache access logs and auth logs to detect abnormal logins, suspicious payloads, and unusual request patterns.
- Monitor for:
  - repeated login attempts
  - unusual query strings indicating injection attempts
  - suspicious user agents or automated scanning behavior
- Review cron execution logs and integrity changes to privileged scripts.

5.2 Containment

- Isolate the affected host from the network.
- Disable exposed services (ports 4444/9001) until access controls are added.
- Terminate suspicious sessions and invalidate tokens/cookies.

5.3 Eradication

- Patch code to remove injection and XSS vulnerabilities.
- Replace insecure cryptographic design with secure key generation and storage.
- Remove dangerous sudo rules and rewrite scheduled scripts using absolute paths.
- Rotate all credentials that could have been exposed.

5.4 Recovery

- Restore from a known clean snapshot or backup.
- Re-enable services with hardened configurations:
  - least privilege
  - secure headers (CSP, HttpOnly, SameSite)
  - firewall rules restricting internal services

- Perform post-incident validation using vulnerability scanning and manual tests.

5.5 Lessons learned

- Most critical failures were not “advanced exploits” but weak defaults, missing validation, and poor operational discipline.
- Key management and privileged automation require special attention due to high impact.

6. Critical Reflection

The most valuable learning outcome from building this CTF VM was understanding how multiple small weaknesses can combine into serious compromise paths. A major challenge was ensuring that high-level tasks were difficult but still fair. For example, the privilege escalation high challenge required balancing realism with solvability by ensuring the PATH condition was intentionally vulnerable and repeatable.

Practical difficulties included Linux permission management (writing into /var/www/html required root privileges) and making sure services reliably persisted across reboots (systemd services and cron). The forensics high challenge required careful validation that the evidence file remained a valid image while still containing an extractable archive, which improved my understanding of file formats and data carving.

Ethically, the VM was isolated using host-only networking to avoid accidental scanning of real networks. The project is intended for educational use and demonstrates attack concepts in a controlled environment, reinforcing responsible security practice rather than encouraging misuse.

7. README Instructions

7.1 Setup Steps

1. Import the VM OVA into VirtualBox.
2. Configure network adapter to **Host-Only** (recommended).
3. Start the VM and note the IP address using ip a.
4. From Kali Linux on the same host-only network, verify reachability with ping and nmap.

7.2 VM Credentials and Access Ports

SSH Credentials (for challenges that require login)

- **Username:** ctfuser
- **Password:** S3rv3r_M1gr@t10n!

Key ports (as seen from attacker machine)

- **80/tcp** – Web portal and challenge pages
- **22/tcp** – SSH (used for privilege escalation stages)
- **4444/tcp** – Network medium challenge (banner leak)
- **9001/tcp** – Network high challenge (debug service)

7.3 Notes for Testing and Grading

- Total flags: **10**
- No internet access required
- Designed to be solved from a Kali Linux attacker VM
- All vulnerabilities are intentional and should not be deployed in real environments

8. Player Walkthrough with Screenshots

Step 1: Host Availability Verification (Reconnaissance)

The first step in any attack is to verify whether the target host is reachable on the network.

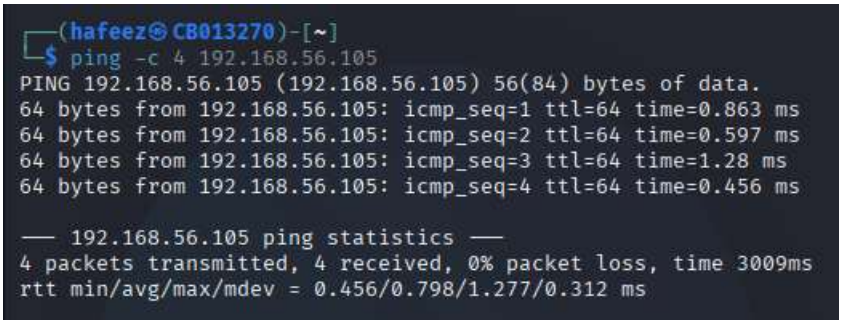
An ICMP ping was sent from the attacker machine (Kali Linux) to the target Ubuntu CTF VM.

Command used:

ping -c 4 192.168.56.105

Observation:

- The host responds successfully to ICMP requests.
- No packet loss was observed.
- This confirms that the target machine is online and reachable.



Step 2: Service Enumeration Using Nmap

After confirming host availability, a port scan was conducted to identify exposed services and potential attack surfaces.

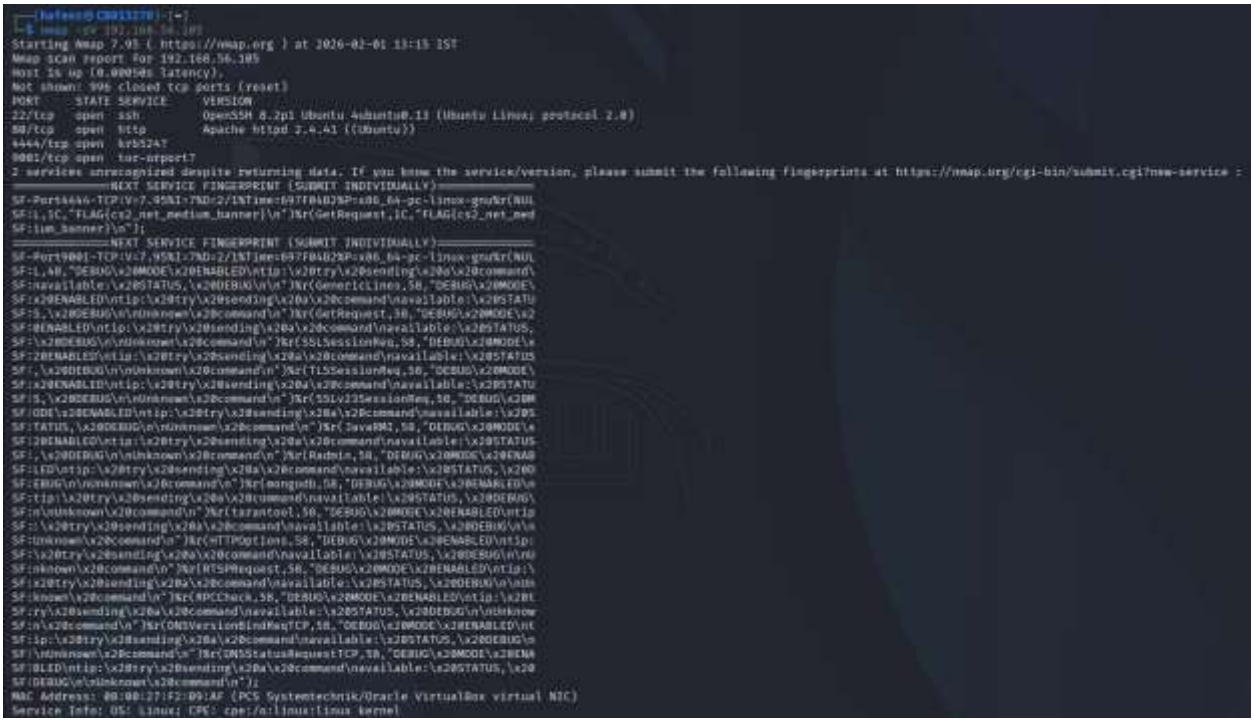
Command used:

nmap -sV 192.168.56.105

Findings:

- Port **22/tcp** → OpenSSH (SSH service)
- Port **80/tcp** → Apache HTTP Server
- Additional non-standard ports were detected (used later in network challenges)

At this stage, the presence of an HTTP service indicates a potential web-based attack vector.



Step 3: Initial Web Enumeration

With port 80 open, the target was accessed through a web browser.

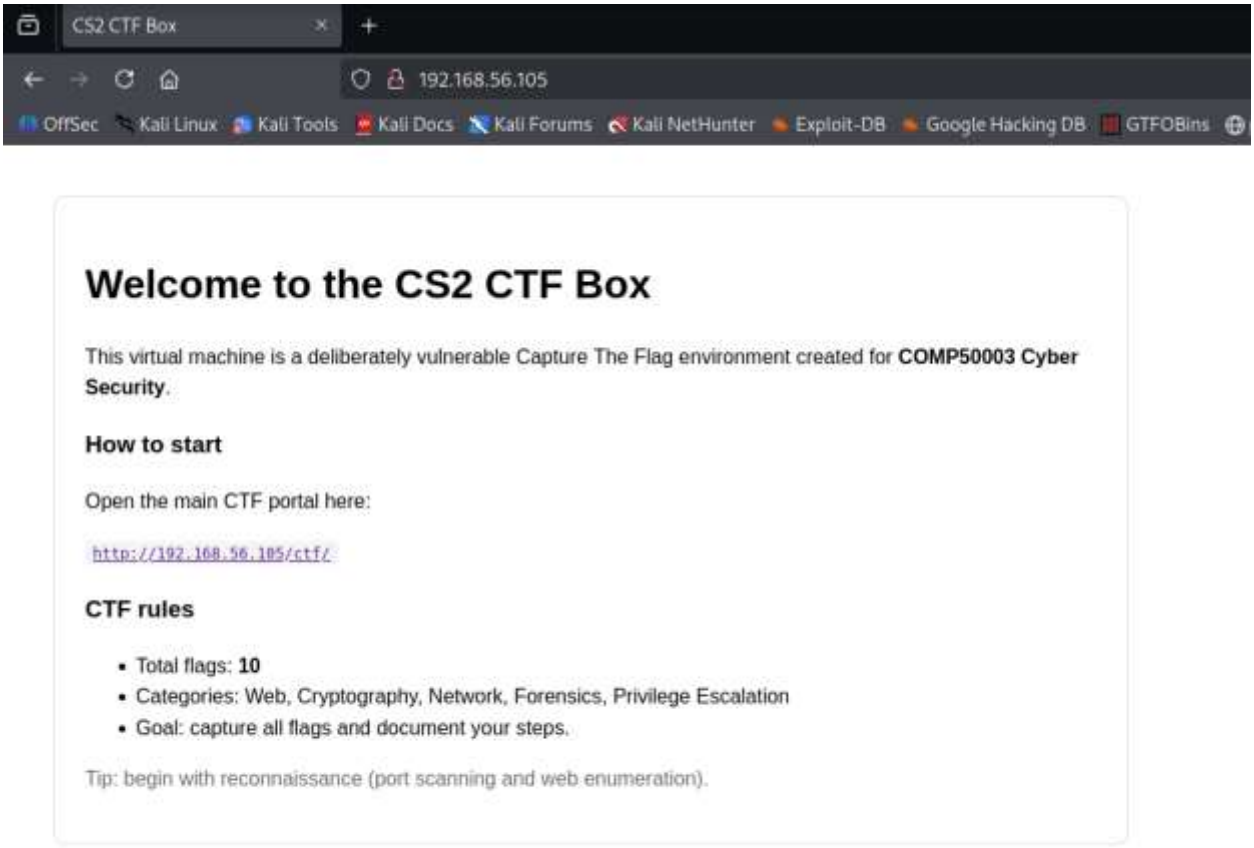
URL accessed:

http://192.168.56.105

Observation:

- A custom landing page titled **“Welcome to the CS2 CTF Box”** is displayed.
- The page provides clear instructions to begin the challenge.
- A link to the main CTF portal is provided.

This confirms that the attack path is intentionally web-driven.



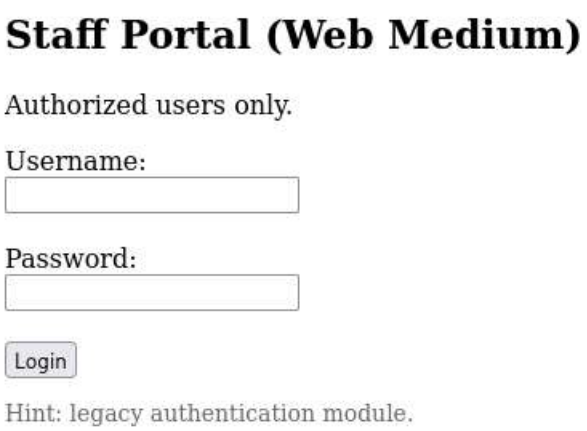
Step 4: Accessing the CTF Portal

Following the instructions on the landing page, the main CTF portal was accessed.

URL accessed:

http://192.168.56.105/ctf/

From the portal, the **Web Medium** challenge is visible and accessible.



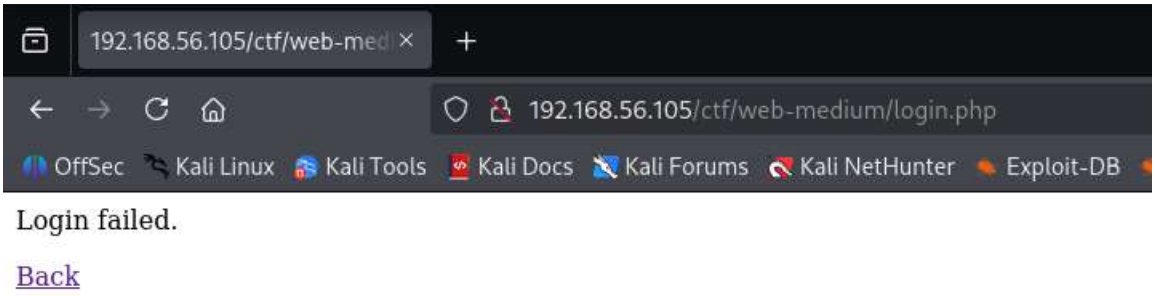
Step 5: Identifying Authentication Mechanism Weakness

The Web Medium challenge presents a login form with a hint:

"legacy authentication module"

This suggests the use of an outdated or insecure authentication mechanism, commonly associated with vulnerabilities such as SQL Injection.

An initial login attempt using random credentials results in a login failure message, confirming server-side validation.



Step 6: Testing for SQL Injection

Based on the hint and behavior of the login form, SQL Injection was tested using a classic authentication bypass payload.

Payload used in the Username field:

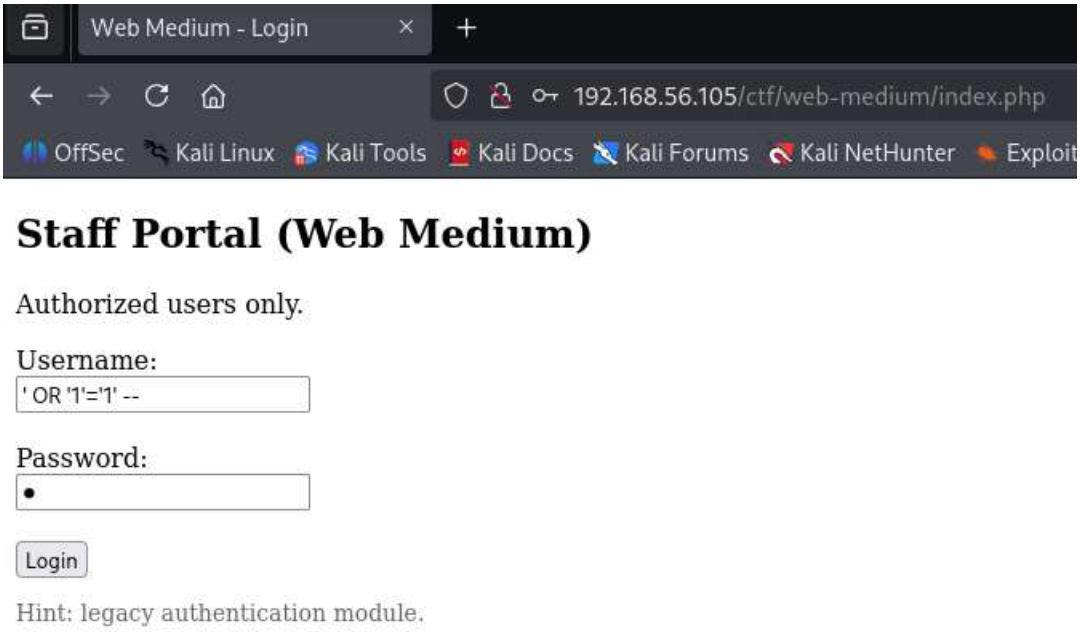
' OR '1'='1' --

Password field:

- Any value = a

Explanation:

- ' OR '1'='1' forces the SQL condition to always evaluate as true.
- -- comments out the remainder of the SQL query.
- This technique exploits improperly sanitized user input in SQL queries.

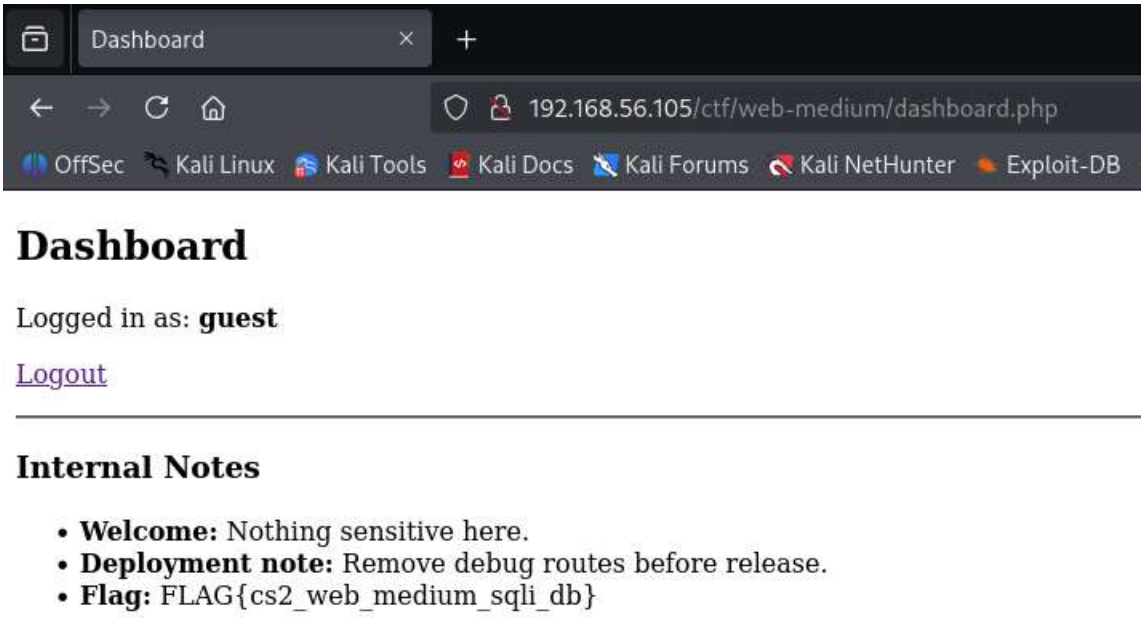


Step 7: Authentication Bypass and Dashboard Access

After submitting the malicious payload, the application grants access without valid credentials.

Observation:

- The user is logged in as **guest**.
- This confirms a successful SQL Injection authentication bypass.
- Sensitive internal content is now accessible.



Step 8: Flag Discovery

Within the dashboard, internal notes are displayed. Among them is a clearly exposed flag intended to confirm challenge completion.

Flag found:

FLAG{cs2_web_medium_sqli_db}

This flag confirms successful exploitation of the Web Medium challenge.

Step 9: Summary of the Exploit

- The application used an insecure, legacy authentication mechanism.
- User input was directly embedded into SQL queries without sanitization.
- This allowed a classic SQL Injection attack to bypass authentication.
- Unauthorized access led directly to disclosure of sensitive information (the flag).

Impact:

- Authentication bypass
- Exposure of internal system data
- Demonstrates a real-world, high-impact web vulnerability

Step 10: Directory Enumeration of the Web Application

After completing the web-based authentication bypass, the attacker continues enumerating the web server to identify additional hidden directories and challenge areas.

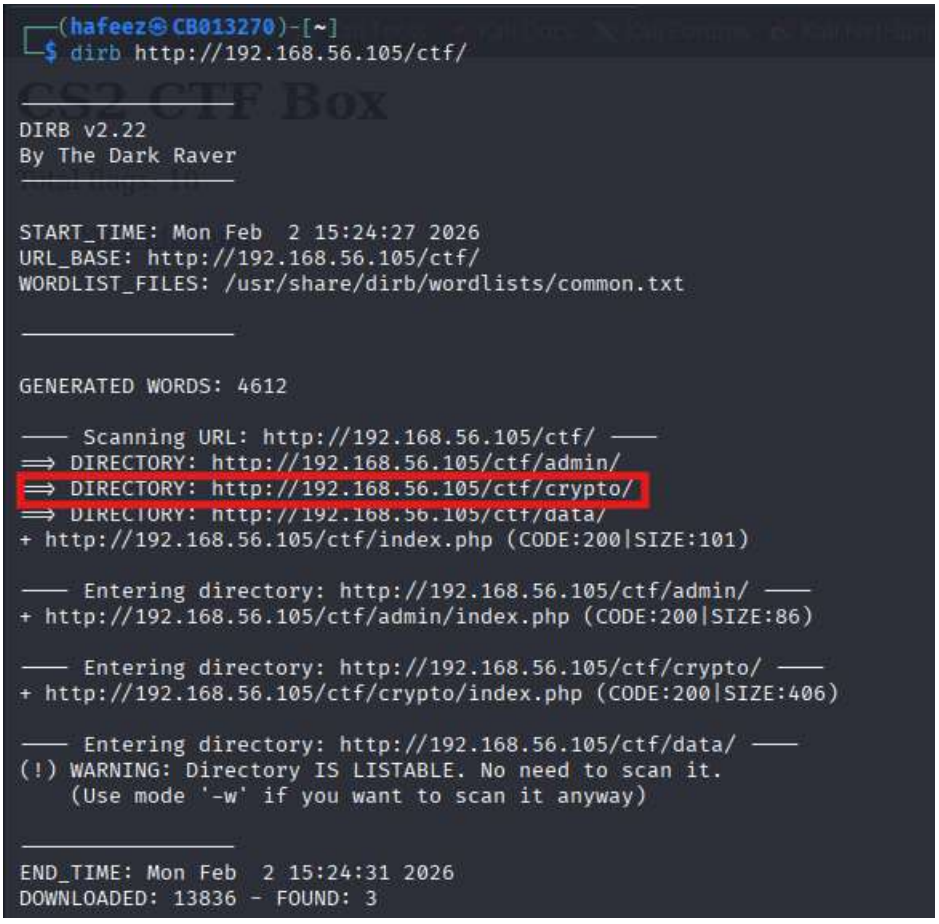
Since the landing page indicates multiple challenge categories, directory brute-forcing is used to locate them.

Command used:

dirb http://192.168.56.105/ctf/

Purpose:

- Identify hidden or unlinked directories
- Discover additional challenge paths not directly linked from the main page



Step 11: Discovery of the Cryptography Challenge Directory

The directory enumeration reveals multiple directories, including:

- /ctf/admin/
- /ctf/crypto/
- /ctf/data/

The presence of the /ctf/crypto/ directory indicates a cryptography-related challenge area.

Step 12: Accessing the Cryptography Challenge Page

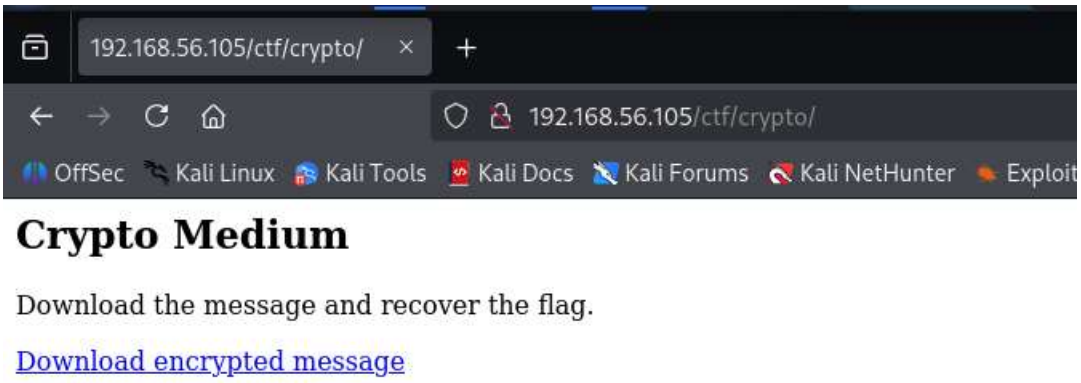
After identifying the directory, the attacker manually navigates to the cryptography challenge.

URL accessed:

http://192.168.56.105/ctf/crypto/

Observation:

- The page is titled **Crypto Medium**
- Instructions indicate that an encrypted message must be downloaded and decrypted



Step 13: Inspecting Page Source for Hidden Hints

Before attempting brute-force decryption, the attacker inspects the page source for developer comments or hidden clues.

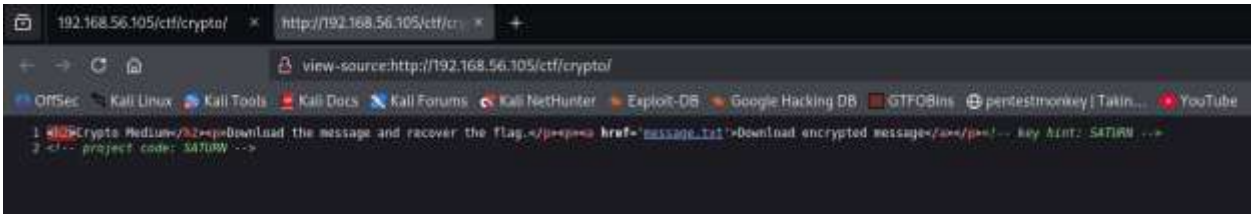
Action performed:

- Right-click → *View Page Source*

Finding:

- An HTML comment reveals a key hint:
- <!-- key hint: SATURN -->
- <!-- project code: SATURN -->

This confirms that the encryption relies on a shared secret key.



Step 14: Downloading the Encrypted Message

Using the link provided on the cryptography page, the encrypted message file is accessed.

URL accessed:

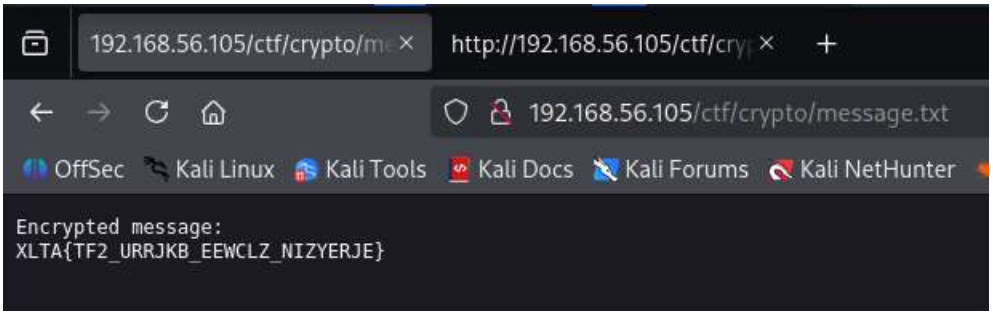
http://192.168.56.105/ctf/crypto/message.txt

Encrypted content:

Encrypted message:

XLTA{TF2_URRJKB_EEWCLZ_NIZYERJE}

The message resembles a flag structure but is clearly encrypted.



Step 15: Identifying the Encryption Algorithm

Based on the following indicators:

- Use of a keyword (SATURN)
- Alphabetic substitution pattern
- Preserved flag format

The encryption method is identified as a Vigenère cipher, a classical polyalphabetic cipher vulnerable to key reuse and leakage.

Step 16: Decrypting the Message

To decrypt the message, a cryptographic analysis tool is used.

Tool used:

- CyberChef

Operation:

- Vigenère Decode

Inputs:

- Ciphertext:
- XLTA{TF2_URRJKB_EEWCLZ_NIZYERJE}
- Key:
- SATURN

Decryption result:

FLAG{CS2_CRYPTO_MEDIUM_VIGENERE}

Step 17: Flag Capture

The decrypted output reveals the cryptography medium flag.

Flag captured:

FLAG{CS2_CRYPTO_MEDIUM_VIGENERE}

This confirms successful exploitation of the cryptography medium challenge.

Step 18: Summary of the Vulnerability

- A legacy cryptographic algorithm (Vigenère cipher) was used
- The encryption key was reused and exposed in client-side code
- Such weaknesses are common in legacy systems and poorly designed encryption schemes
- An attacker can recover sensitive data without brute-force or advanced cryptanalysis

Step 19: Identifying Network-Level Attack Surface

After completing the cryptography challenge, the attacker returns to network-level reconnaissance to identify any services that may expose sensitive information through misconfiguration.

A service version scan is performed to enumerate all open TCP ports and running services.

Command used:

nmap -sV 192.168.56.105

Purpose:

- Identify open ports beyond standard web services
- Detect misconfigured or custom network services
- Gather banner and protocol information

Step 20: Analysis of Nmap Results

The scan reveals the following open services:

Port	Service	Observation
22	SSH	Standard remote access
80	HTTP	Apache web server
4444	Unknown (krb524?)	Suspicious non-standard service
9001	Unknown (tor-orport?)	Custom service, requires further analysis

Port **4444** stands out because:

- It is not a commonly exposed service
- Nmap is unable to correctly identify the application
- Banner data appears to be returned by the service

Step 21: Manual Interaction With Suspicious Service

To further investigate port **4444**, a manual TCP connection is initiated using Netcat.

Command used:

nc 192.168.56.105 4444

Step 22: Banner Disclosure and Flag Exposure

Upon connecting to the service, the server immediately responds with a plaintext banner containing a flag.

Observed output:

FLAG{cs2_net_medium_banner}

This confirms that the service is misconfigured and directly exposes sensitive information without authentication.

Step 23: Flag Capture

The network medium flag is successfully retrieved.

Flag captured:

FLAG{cs2_net_medium_banner}

Step 24: Vulnerability Explanation

This vulnerability is classified as **Information Disclosure via Network Service Banner**.

Key issues:

- Custom service running on a non-standard port
- Debug or testing output left enabled
- No authentication or access control
- Sensitive data returned immediately upon connection

Such issues are common in:

- Development or staging environments
- Poorly hardened servers
- Custom internal services accidentally exposed to external networks

Step 25: Security Impact

An attacker does not require:

- Credentials
- Exploitation tools
- Protocol knowledge

Simple network enumeration is sufficient to retrieve sensitive information, making this vulnerability **low complexity but high impact**.

Step 26: Identifying the Next Network Vulnerability

During the earlier service enumeration, a second unidentified service was observed running on **port 9001**.
Since this port was already discovered in the Nmap scan, the attacker proceeds directly to interact with it.

The service description suggests a custom or internal application, which often indicates debugging or management functionality.

Step 27: Connecting to the Service on Port 9001

To investigate the service, a manual TCP connection is established using Netcat.

Command used:

nc 192.168.56.105 9001

Upon connection, the service responds with the following message:

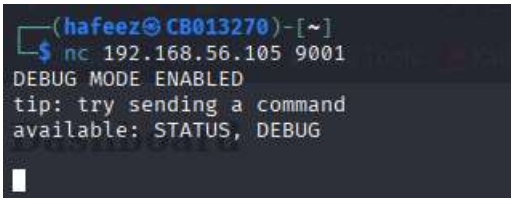
DEBUG MODE ENABLED

tip: try sending a command

available: STATUS, DEBUG

This confirms that:

- The service is running in **debug mode**
- It accepts interactive commands
- No authentication is required



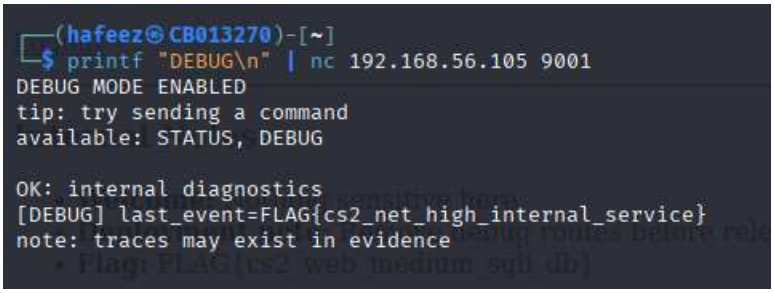
Step 28: Interacting With the Debug Service

Based on the service prompt, the attacker attempts to interact with the service by sending supported commands.

To do this, input is piped directly into Netcat.

Command used:

printf "DEBUG\n" | nc 192.168.56.105 9001



Step 29: Debug Information Disclosure

After sending the DEBUG command, the service responds with internal diagnostic information:

OK: internal diagnostics

[DEBUG] last_event=FLAG{cs2_net_high_internal_service}

note: traces may exist in evidence dumps

This output confirms that sensitive internal data is being exposed through the debug interface.

Step 30: Flag Capture

The network high flag is successfully retrieved.

Flag captured:

FLAG{cs2_net_high_internal_service}

Step 31: Vulnerability Explanation

This vulnerability is classified as **Exposed Internal Debug Service**.

Key weaknesses:

- Debug mode enabled in a production-like environment
- No authentication or access control

- Direct disclosure of sensitive internal state
- Custom protocol accessible over a raw TCP socket

Such services are commonly left behind during development or testing and can lead to severe information leakage.

Step 32: Security Impact

An attacker can:

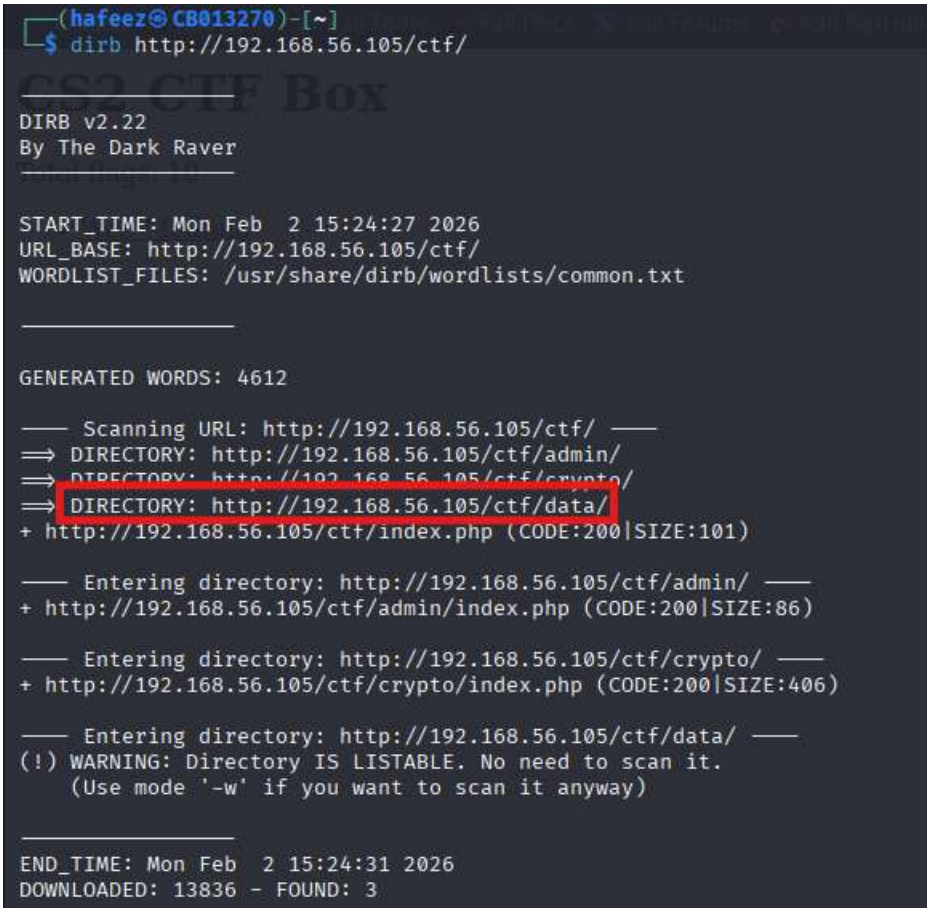
- Enumerate internal application state
- Leak sensitive debugging data
- Potentially trigger undocumented commands

No exploitation framework is required - a basic TCP client is sufficient.

Step 33: Identifying a Forensics Entry Point via Directory Enumeration

After completing the network exploitation challenges, the attacker returns to previously collected directory enumeration results to identify any unvisited paths that may contain forensic artifacts.

The earlier dirb scan against /ctf/ revealed a directory that had not yet been analyzed in detail.



While /ctf/data/ exists, further manual exploration reveals that the actual forensic evidence is exposed under a different directory: **/ctf/evidence/**, which is not directly linked from the main portal.

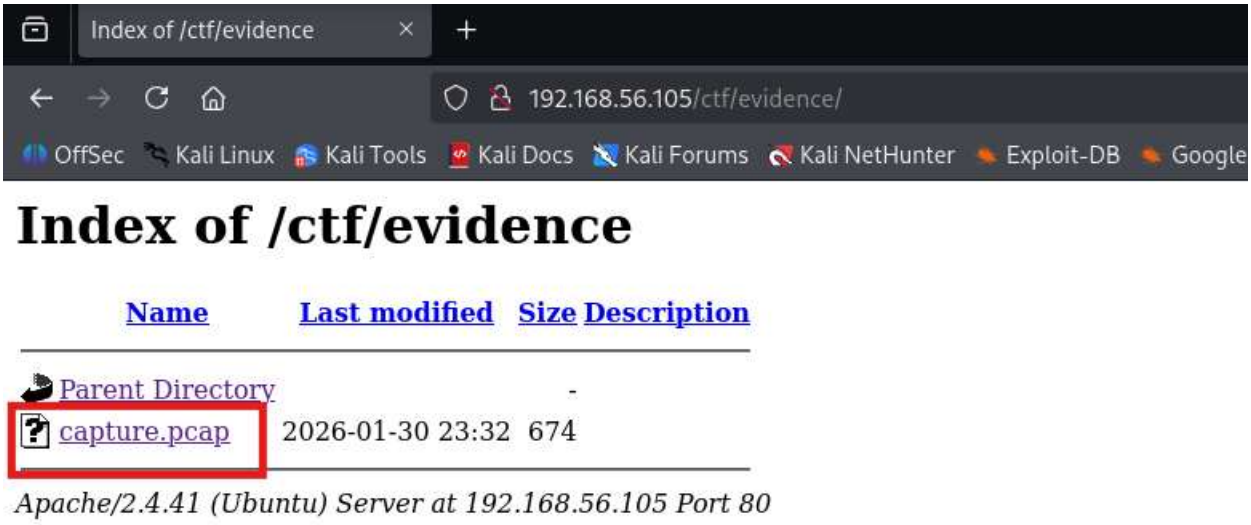
Step 34: Discovering an Open Evidence Directory

The attacker manually navigates to the suspected evidence directory.

URL accessed:

http://192.168.56.105/ctf/evidence/

The server responds with an **Apache directory listing**, indicating that directory indexing is enabled.



Step 35: Identifying a Packet Capture File

Within the directory listing, a packet capture file is exposed:

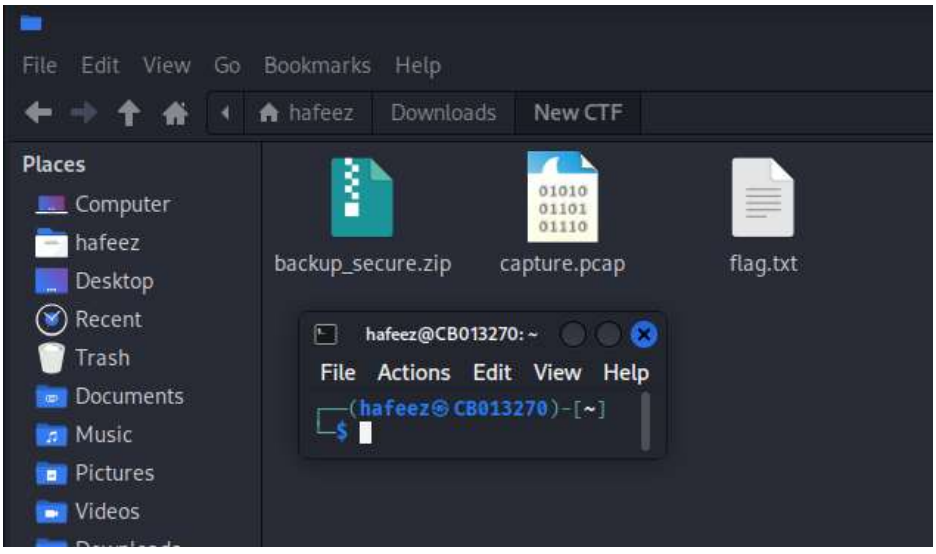
capture.pcap

PCAP files commonly contain recorded network traffic and are frequently used in forensic investigations to analyze past communications.

Step 36: Downloading the Forensic Artifact

The attacker downloads the packet capture file for offline analysis.

The file is saved locally on the attacker machine for inspection using forensic tools.



Step 37: Opening the PCAP File in Wireshark

To analyze the captured network traffic, the attacker opens the file using Wireshark.

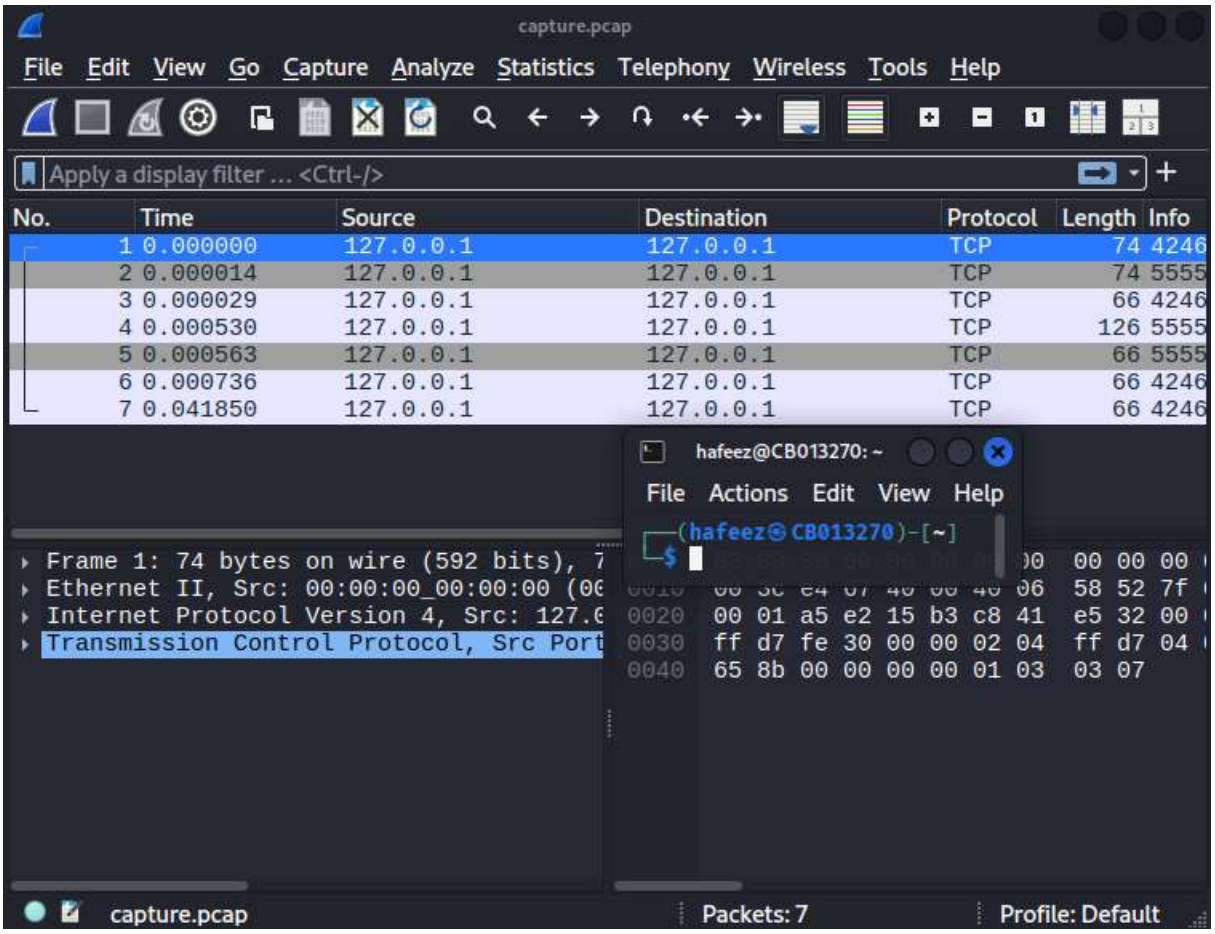
Tool used:

- Wireshark

Action performed:

- Open capture.pcap

The packet list shows a small number of TCP packets, suggesting a short recorded session.



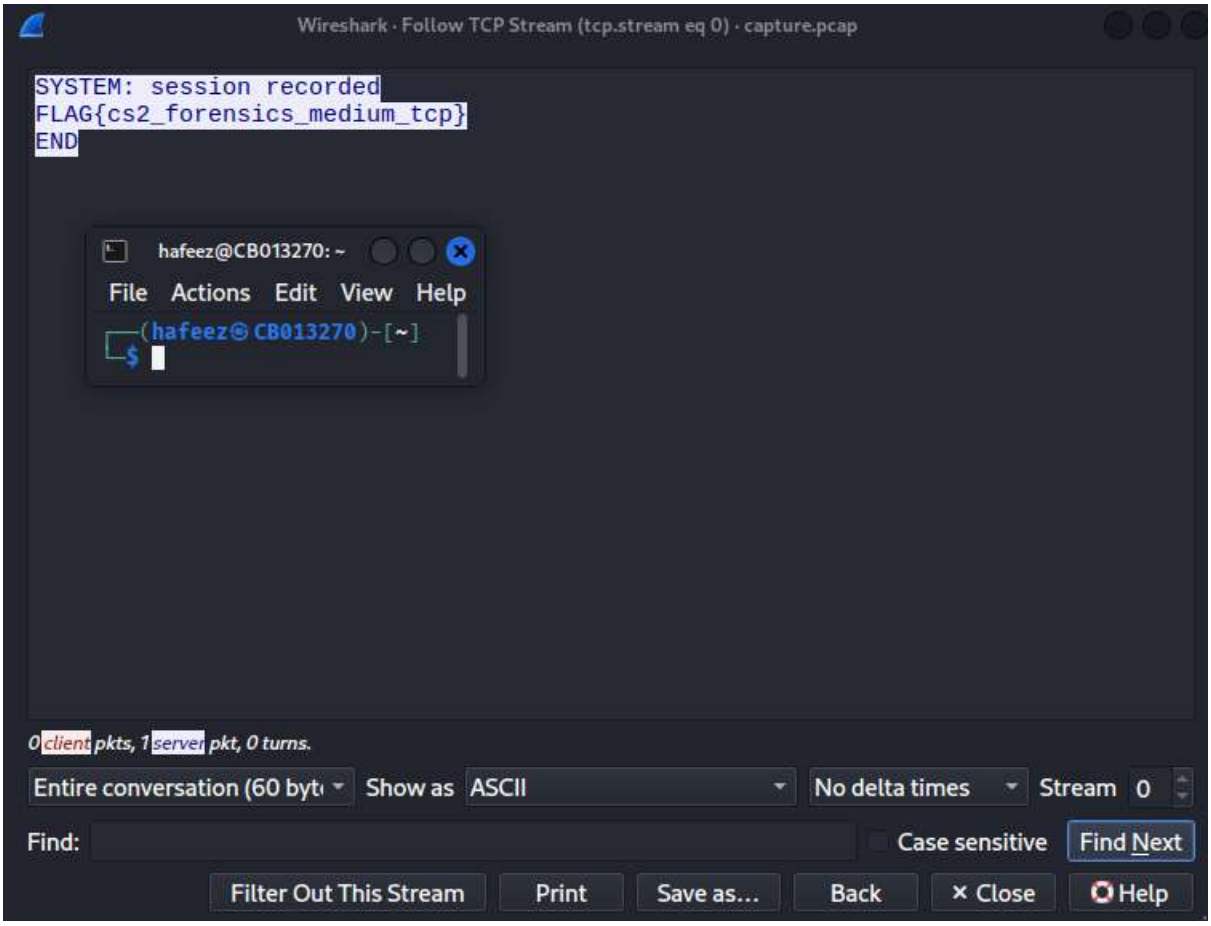
Step 38: Inspecting TCP Conversations

To reconstruct the communication session, the attacker follows the TCP stream.

Action performed in Wireshark:

Right-click packet → Follow → TCP Stream

This reconstructs the full data exchange between the client and server in human-readable form.



Step 39: Recovering the Flag from Network Traffic

The reconstructed TCP stream reveals plaintext data transmitted during the session:

SYSTEM: session recorded

FLAG{cs2_forensics_medium_tcp}

END

This indicates that sensitive information was transmitted **unencrypted over TCP** and captured in the packet trace.

Step 40: Flag Capture

The forensic medium flag is successfully recovered.

Flag captured:

FLAG{cs2_forensics_medium_tcp}

Step 41: Vulnerability Explanation

This challenge demonstrates **insecure transmission of sensitive data over plaintext network protocols**.

Key issues identified:

- Sensitive data transmitted without encryption
- Network traffic recorded and stored without access control
- PCAP file publicly accessible via web server directory listing

Such weaknesses are common in:

- Misconfigured servers
- Debugging or testing environments
- Poorly secured logging and monitoring systems

Step 42: Security Impact

An attacker with access to captured traffic can:

- Reconstruct full sessions
- Extract sensitive information
- Bypass authentication or confidentiality controls

This highlights the importance of:

- Using encrypted protocols (TLS)
- Restricting access to forensic artifacts
- Disabling directory listing in production environments

Step 43: Identifying a Hint for Further Forensic Analysis

After completing the **Network High** challenge, the attacker carefully reviews the output returned by the exposed debug service.

The debug response contained the following message:

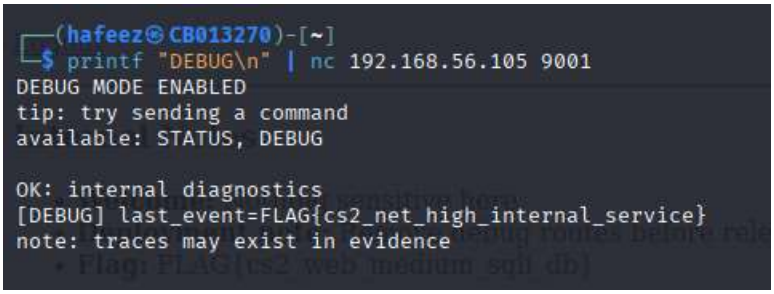
“note: traces may exist in evidence dumps”

This message strongly implies that:

- Additional forensic artifacts exist on the system

- These artifacts may not be directly linked or indexed
- The attacker should search for **evidence-related resources**

This serves as a **narrative and technical hint** to continue forensic investigation.



Step 44: Investigating Forensics-Specific Paths

Based on the hint about *evidence dumps*, the attacker attempts to manually enumerate common forensic-related paths rather than relying solely on automated directory brute-forcing.

following URL:

http://192.168.56.105/ctf/forensics-high/

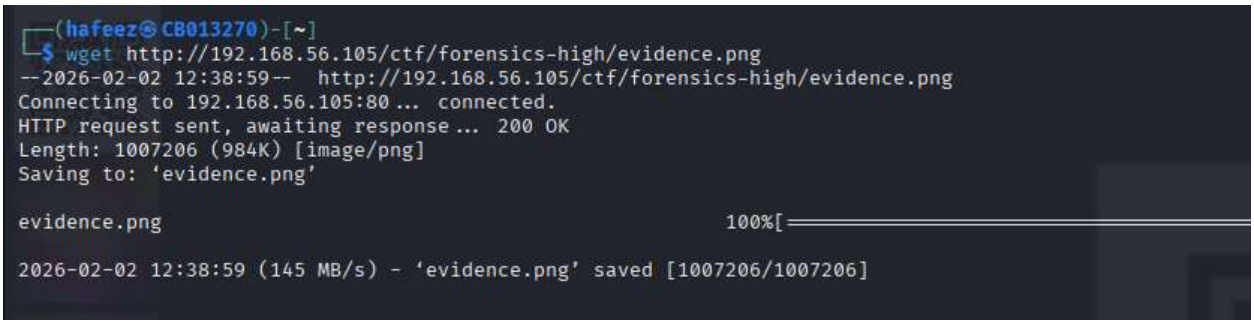
This directory is not listed by dirb, indicating that it may be intentionally hidden or excluded from wordlists.

Step 45: Discovering the Forensic Evidence File

Accessing the directory reveals a downloadable file:

evidence.png

This file appears to be an image, but given the forensic context, it is treated as a potential container for hidden data.



Step 46: Verifying File Type

Before analyzing the file, the attacker verifies its format.

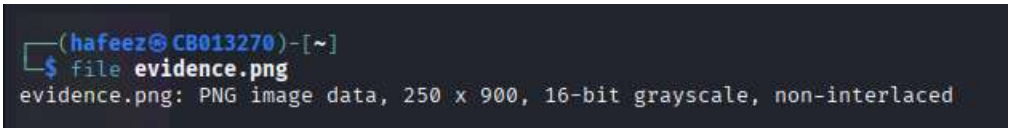
Command used:

file evidence.png

Result:

PNG image data, 250 x 900, 16-bit grayscale

While it is a valid PNG file, forensic challenges often embed additional data beyond the visible image.



Step 47: Searching for Embedded File Signatures

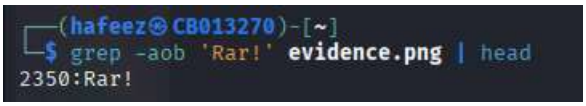
To check for hidden content, the attacker searches the image for known file signatures.

RAR archives typically contain the string Rar!.

Command used:

grep -aob 'Rar!' evidence.png | head

The output indicates that a RAR file signature exists at byte offset 2350.



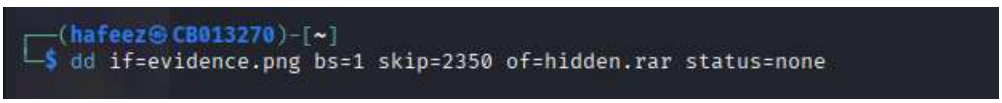
Step 48: Extracting the Embedded Archive

Using the identified offset, the attacker extracts the embedded archive from the image.

Command used:

dd if=evidence.png bs=1 skip=2350 of=hidden.rar status=none

This operation carves the hidden archive from the image file.



Step 49: Extracting the RAR Archive

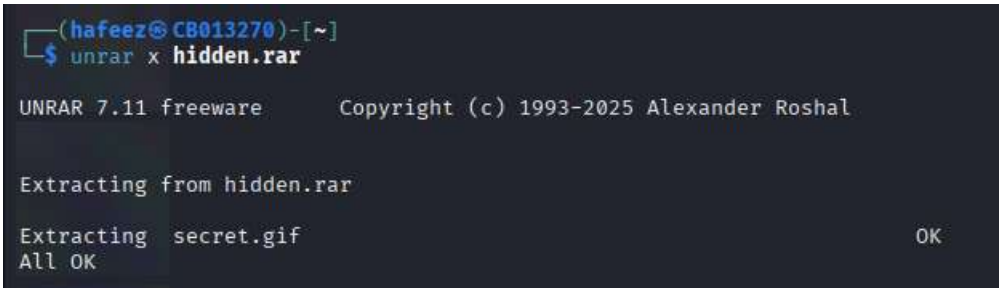
The extracted archive is then decompressed.

Command used:

unrar x hidden.rar

The extraction reveals a file named:

secret.gif



Step 50: Analyzing the Extracted GIF File

The file type is verified:

Command used:

file secret.gif

Result:

GIF image data, version 89a

Animated GIFs often contain multiple frames, which can be used to hide information across frames.

Step 51: Extracting Frames from the GIF

To analyze each frame individually, the attacker extracts all frames from the GIF.

Commands used:

mkdir frames

convert secret.gif frames/frame_%02d.png

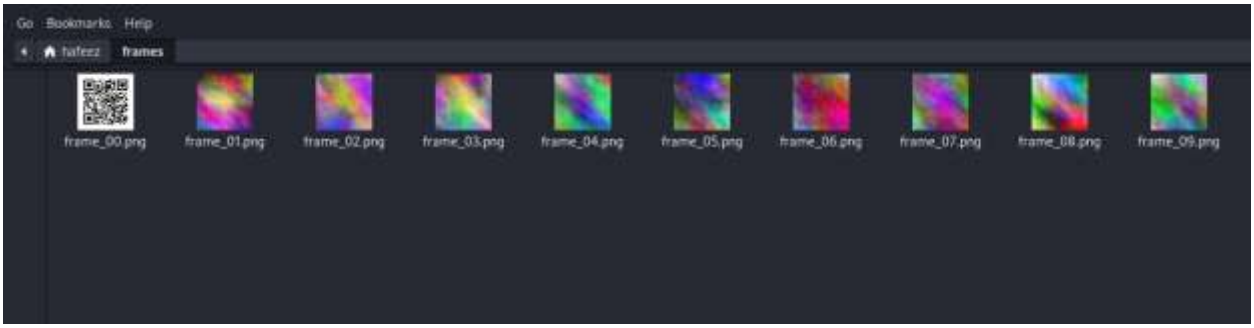
ls frames | wc -l

This reveals 10 frames, confirming that the GIF is animated.

Step 52: Identifying the Hidden QR Code

Among the extracted frames, one frame clearly contains a **QR code**, while the others appear as noise or decoys.

The QR code is scanned using an image recognition or QR scanning tool.

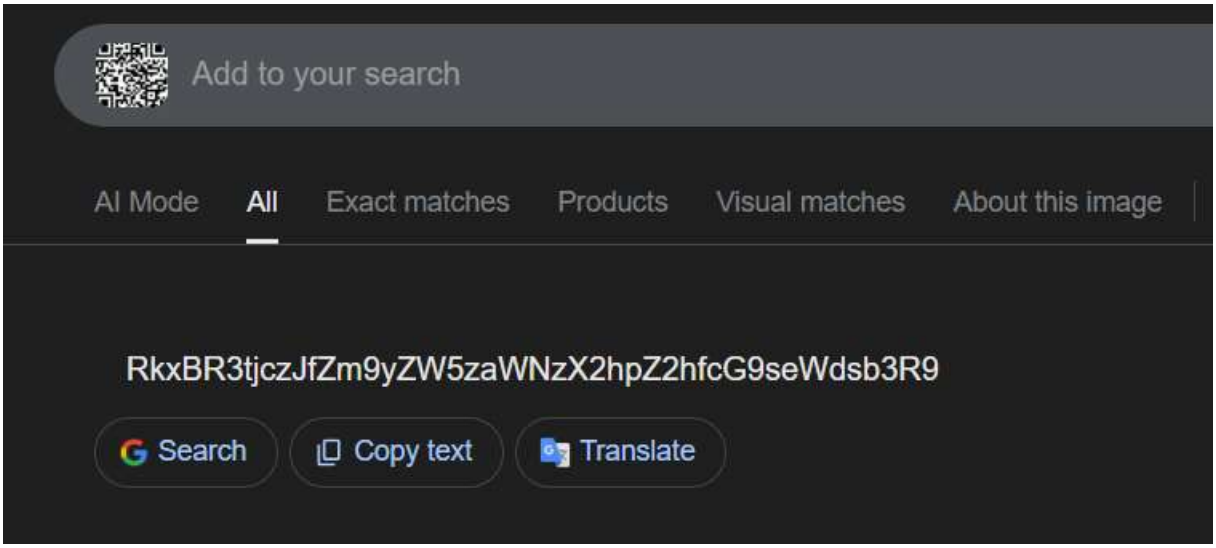


Step 53: Extracting Encoded Data from the QR Code

The QR code resolves to the following encoded string:

RkxBR3tjczJfZm9yZW5zaWNzX2hpZ2hfcG9seWdsb3R9

This string appears to be Base64-encoded.



Step 54: Decoding the Base64 Payload

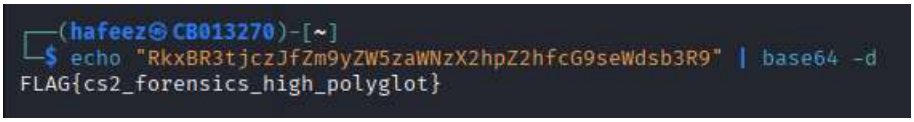
To decode the extracted string, the attacker uses Base64 decoding.

Command used:

echo "RkxBR3tjczJfZm9yZW5zaWNzX2hpZ2hfcG9seWdsb3R9" | base64 -d

Decoded output:

FLAG{cs2_forensics_high_polyglot}



Step 55: Flag Capture

The forensics high flag is successfully recovered.

Flag captured:

FLAG{cs2_forensics_high_polyglot}

Step 56: Vulnerability Explanation

This challenge demonstrates **polyglot file abuse and forensic data hiding techniques**, including:

- Embedding archives inside image files
- Bypassing detection by using valid file headers
- Leveraging multi-frame GIFs to hide secondary payloads

- Encoding sensitive data using Base64

Such techniques are commonly used in:

- Malware droppers
- Data exfiltration
- Steganography-based persistence

Step 57: Security Impact

If attackers gain access to forensic artifacts:

- Hidden payloads can bypass security controls
- Sensitive data can be covertly exfiltrated
- Traditional file-type validation becomes ineffective

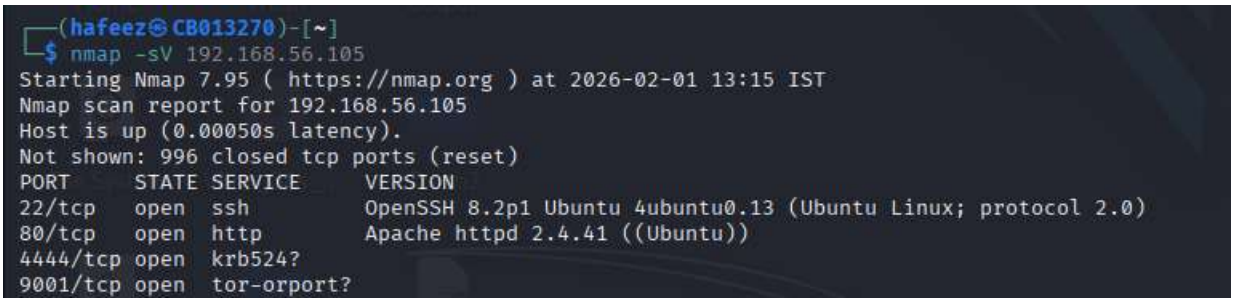
Step 58: Identifying Local Access via SSH

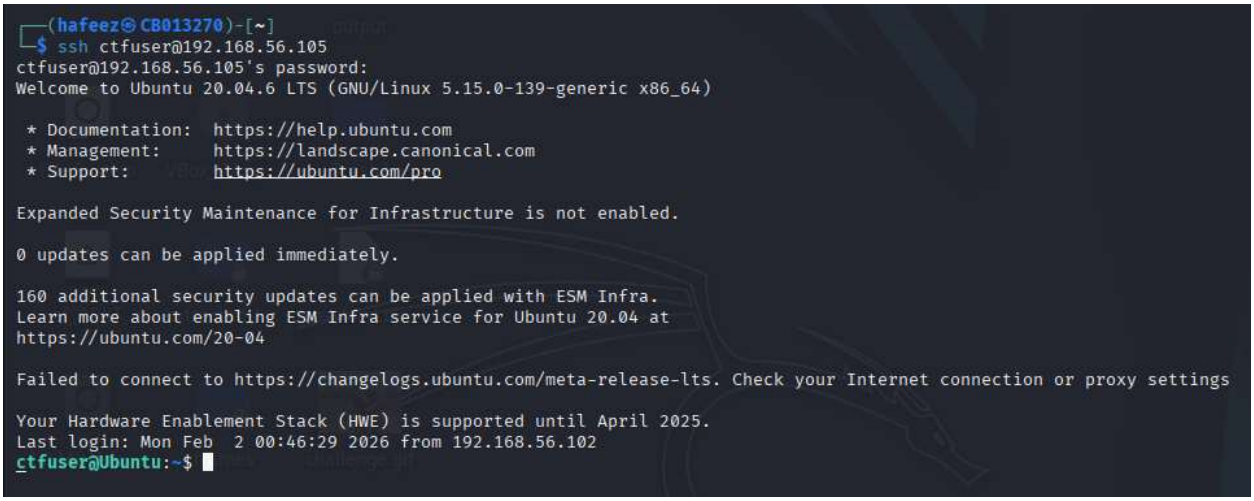
From earlier reconnaissance, SSH was identified as an open service on port 22.
At this stage, the attacker attempts to gain **local access** to the system using known credentials provided as part of the CTF environment.

Command used:

ssh ctfuser@192.168.56.105

After entering the password, SSH access is successfully obtained.





Step 59: Enumerating Privilege Escalation Opportunities

Once logged in as a low-privileged user, the attacker checks for misconfigured sudo permissions that could allow privilege escalation.

Command used:

sudo -l

Step 60: Analyzing Sudo Permissions

The sudo configuration reveals the following critical misconfiguration:

(ctfuser) NOPASSWD: /usr/bin/less

This means:

- ctfuser can execute /usr/bin/less **as root**
- No password is required
- less is an interactive program capable of spawning a shell

This is a well-known privilege escalation vector documented in **GTFOBins**.

Step 61: Exploiting less to Gain Root Access

The attacker executes less with elevated privileges:

sudo /usr/bin/less /root/flag_priv_medium.txt

Since less runs as root, it allows command execution from within its interactive prompt.

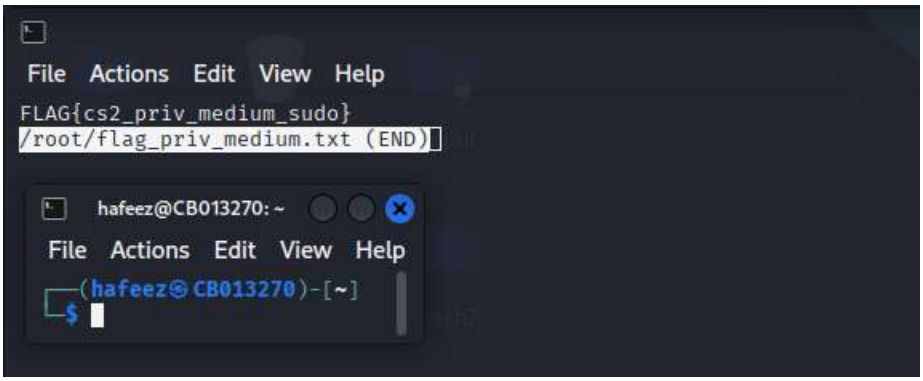
Inside less, the attacker spawns a shell by typing:

!sh

This results in a **root shell**, inheriting root privileges.

Step 62: Accessing Root-Only Files

With root privileges obtained, the attacker reads the flag file located in the /root directory.



Flag captured:

FLAG{cs2_priv_medium_sudo}

Step 63: Vulnerability Explanation

This vulnerability is classified as **Insecure Sudo Configuration**.

Key issues:

- Granting NOPASSWD sudo access to interactive binaries
- Failure to restrict commands to non-interactive, fixed-function binaries
- Lack of command execution restrictions

Tools like less, vim, nano, and man are **not safe** to grant sudo access to, as they allow shell escapes.

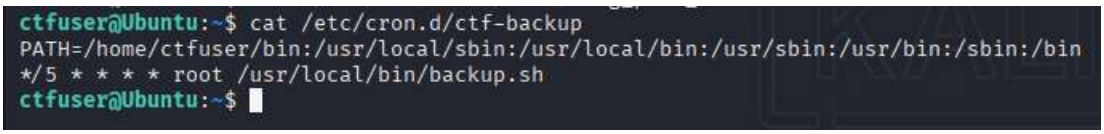
Step 64: Security Impact

An attacker with low-privileged access can:

- Escalate to root instantly
- Access sensitive files
- Modify system configurations
- Establish persistence

This is a **high-impact misconfiguration**, even though exploitation is simple.

Step 65: Identifying Scheduled Tasks as an Attack Vector



After successfully logging in as the low-privileged user (ctfuser), the next step was to look for automated processes running with elevated privileges. Scheduled tasks such as cron jobs are common privilege escalation vectors if they execute scripts insecurely.

The file /etc/cron.d/ctf-backup was inspected, revealing a cron job that runs every 5 minutes as the root user.

This cron job executes the script:

/usr/local/bin/backup.sh

The presence of a root-owned scheduled task immediately indicates a potential privilege escalation opportunity.

Step 66: Analyzing the Root Backup Script

The contents of the backup script were reviewed to understand its behavior.

The script performs the following actions:

- Uses the tar command to archive the /etc directory
- Writes the archive to /tmp/backup.tar
- Does not use absolute paths for the tar binary

This is a **critical security flaw**. When commands like tar are executed **without an absolute path**, the system searches for the binary based on the PATH environment variable. If the PATH can be influenced, a malicious replacement binary can be executed instead.

Step 67: Exploiting PATH Hijacking via Cron

```
ctfuser@Ubuntu:~$ cat /usr/local/bin/backup.sh
#!/bin/bash
echo "[*] Running daily backup ..."
tar -cf /tmp/backup.tar /etc >/dev/null 2>&1
echo "[*] Backup complete"
ctfuser@Ubuntu:~$
```

From the cron configuration, it was observed that /bin appears early in the PATH. This allows an attacker to place a **malicious executable named tar** in /bin, which will be executed instead of the legitimate /usr/bin/tar.

A malicious tar script was created with the following behavior:

- Copies a root-only flag file into /tmp
- Changes its permissions to make it readable by any user

This effectively abuses the root cron job to execute attacker-controlled code with **root privileges**.

Step 68: Making the Malicious Binary Executable

```
ctfuser@Ubuntu:~$ mkdir -p ~/bin
ctfuser@Ubuntu:~$ cat <<'EOF' > ~/bin/tar
> #!/bin/bash
> /bin/cp /root/flag_priv_high.txt /tmp/flag_priv_high.txt
> /bin/chmod 644 /tmp/flag_priv_high.txt
> EOF
ctfuser@Ubuntu:~$ chmod +x ~/bin/tar
ctfuser@Ubuntu:~$
```

After creating the malicious script, executable permissions were applied to ensure it could be executed by the cron job.

Once this step was completed, no further action was required from the attacker. The cron job would automatically execute the malicious binary within the next scheduled interval.

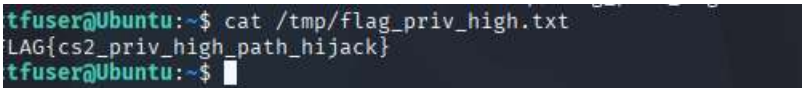
Step 69: Verifying Privilege Escalation Success

```
ctfuser@Ubuntu:~$ ls -l /tmp/flag_priv_high.txt
-rw-r--r-- 1 root root 32 සෙ     2 00:55 /tmp/flag_priv_high.txt
```

After waiting for the cron job to execute, the /tmp directory was checked. A new file, created by the malicious tar script, was present and owned by root.

This confirmed that **arbitrary command execution as root** had been successfully achieved via PATH hijacking.

Step 70: Capturing the Final Privilege Escalation Flag



The contents of the file were read, revealing the final flag for this challenge:

FLAG{cs2_priv_high_path_hijack}

This confirms successful exploitation of a **high-impact privilege escalation vulnerability** caused by:

- Insecure cron job configuration
- Use of non-absolute binary paths
- Writable directories included in the execution PATH

Summary of Privilege Escalation (High)

- **Technique Used:** PATH Hijacking via root cron job
- **Misconfiguration:** Cron script executed system binaries without absolute paths
- **Impact:** Full root-level command execution
- **Flag Captured:** FLAG{cs2_priv_high_path_hijack}